Classes and Objects

- Classes and Objects in Python
- Object-Oriented Programming Concepts

What are Classes and Objects?

- World is **object-oriented**
- Full of **objects** like **Sparrow, Rose, Guitar, Keyboard**, etc.
- Each **object** is a **specific instance** of a **class**
- E.g. Sparrow is a specific instance of a **Bird** class
- Rose is a specific instance of a **Flower** class

Examples of Classes and Objects in Real Life

- **Bird** is a class → Sparrow, Crow, Eagle are **objects** of the Bird class.
- **Player** is a class → Sachin, Rahul, Kapil are **objects** of the Player class.
- **Flower** is a class → Rose, Lily, Gerbera are **objects** of the Flower class.
- **Instrument** is a class → Sitar, Flute are **objects** of the Instrument class.

Object-Oriented Thinking

- A class is a **blueprint** or **template** used to create **objects**.
- A **class describes two things**:
  - The **form** an object created from it will take.
  - The **functionality** it will have.
- E.g., a **Bird class** may specify:
- **Form (Properties)**
  - Weight
  - Color
  - Number of feathers

Object-Oriented Thinking

- **Functionality (Methods)**
  - Flying
  - Hopping
  - Chirping
  - Eating

Class → Generic in nature
Object → Specific in nature

- A class lets us bundle **data and functionality together**.
- When objects like **Sparrow** or **Eagle** are created from the Bird class, the properties will have specific values.
  - E.g: Property **weight** may be **250 grams** for a Sparrow object.
  - The same property may be **10 kg** for an Eagle object.

Classes and Objects in Programming

- In Python, **every type is a class**.
- Examples of classes: int, str, list, tuple, set, dict
- A class has a **name**, whereas objects are **nameless**
- Since objects do not have names, they are referred to using their **memory addresses**
- Example: num = 10
- A **nameless object** of **type int** is **created** in **memory**, and its **address** is stored in **num**.
- Thus, num refers to the object containing value 10.
- Instead of saying that num refers to a nameless int object, we simply say:
- **num is an int object**

Examples of Classes and Objects in Python

- a = 3.14 # a is an object of float class
- s = 'Sudesh' # s is an object of str class
- lst = [10, 20, 30] # lst is an object of list class
- tpl = ('a', 'b', 'c') # tpl is an object of tuple class
- Different objects of the same type may contain **different data but the same methods**.
- Example:
- s1 = 'Rupesh' # s1 is an object of type str
- s2 = 'Geeta' # s2 is an object of type str

Examples of Objects

- s1 = 'Rupesh' # s1 is an object of type str
- s2 = 'Geeta' # s2 is an object of type str
- s1 and s2 contain **different data**
- But both support the same methods such as:
  - upper()
  - lower()
  - capitalize()
- The specific data stored in an object is called:
- **Instance data**
- **Attributes**
- **State of the object**
- The functions defined inside objects are called **methods**.

User-defined Classes

- In addition to providing ready-made classes like **int, str, list, tuple**, Python also allows us to **define our own classes** and create objects from them.
- These are called **user-defined classes**.
- The rules for defining and using user-defined classes are the same as for standard classes.

Class vs Object

- **Class** → Blueprint
- **Object** → Instance created from the class
- The process of creating an object from a class is called **instantiation**
- **Attributes** store **data**
- **Methods** perform **operations** on that data

OOP, Defining a Class

- Python was built as a procedural language
  - OOP exists and works fine, but feels a bit more "tacked on"

- Declaring a class:

```
class name:
    statements
```

Fields

name = value

– Example:

```
class Point:
    x = 0
    y = 0

# main
p1 = Point()
p1.x = 2
p1.y = -5
```

point.py

```
1  class Point:
2      x = 0
3      y = 0
```

Using a Class

`import` **class**

– client programs must import the classes they use

point_main.py

```
from Point import *

# main
p1 = Point()
p1.x = 7
p1.y = -3
...

# Python objects are dynamic (can add fields any time!)
p1.name = "Tyler Durden"
```

- In Python, you can add new attributes anytime!
- Even if name was NOT defined in the class, you can still add it:

Now p1 has:

- x
- y
- name

Object Methods

```
def name(self, parameter, ..., parameter):
    statements
```

- self refers to the current object
- It is similar to this in C++/Java
- **`self`** *must* be the first parameter to any object method
- *must* access the object's fields through the `self` reference
- Each object stores its own data separately

```
class Point:
    def translate(self, dx, dy):
        self.x += dx
        self.y += dy
    ...
```

Write `distance`, `set_location`, and `distance_from_origin` methods.

point.py

```
from math import *

class Point:
    x = 0
    y = 0

    def set_location(self, x, y):
        self.x = x
        self.y = y

    def distance_from_origin(self):
        return sqrt(self.x * self.x + self.y * self.y)

    def distance(self, other):
        dx = self.x - other.x
        dy = self.y - other.y
        return sqrt(dx * dx + dy * dy)
```

```
class Employee:
    def set_data(self, n, a, s):
        self.name = n
        self.age = a
        self.salary = s

    def display_data(self):
        print(self.name, self.age, self.salary)

e1 = Employee()
e1.set_data('Ramesh', 23, 25000)
e1.display_data()

e2 = Employee()
e2.set_data('Suresh', 25, 30000)
e2.display_data()
```